

Analysis of Algorithms - 2025

Algorithm

- Design
- Domain knowledge
- Language
- Hardware, OS
- Analysis

Program

- Implementation
- Programmer
- programming language
- Hardware, OS
- Testing

Priori Analysis

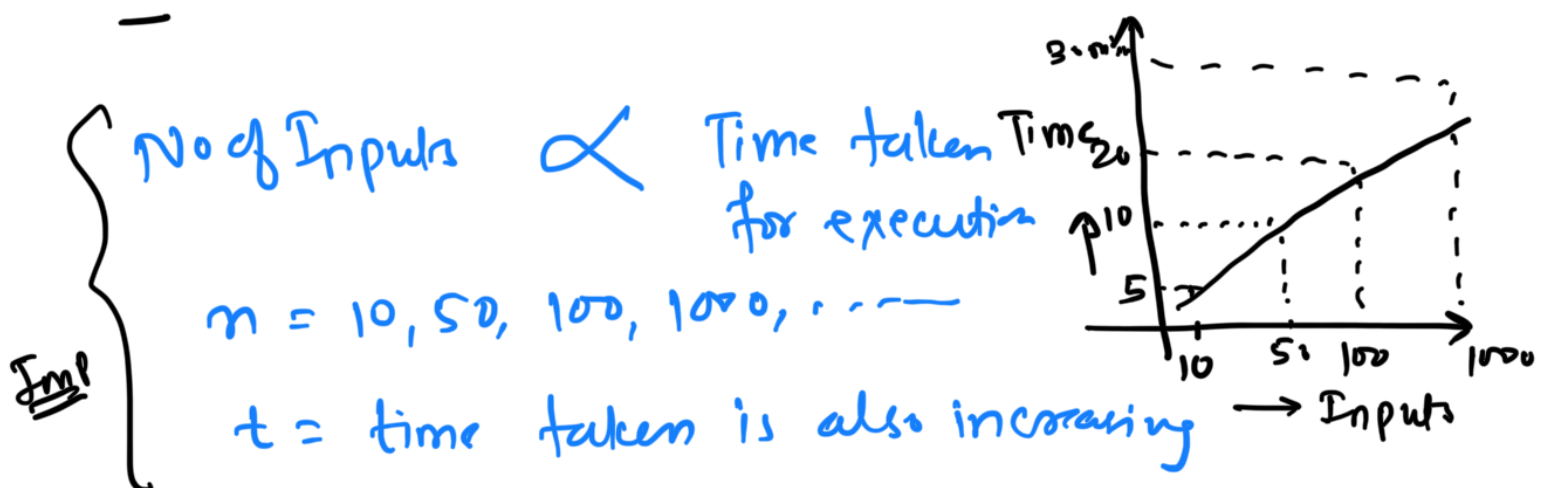
- Algorithms
- Independent of PL
- Independent of HW
- Time & Space

Posterior Analysis

- Program
- Dependent on PL
- Dependent of HW
- Time

Reason -

- helps in evaluating the efficiency of an algorithm before implementation



- Ensure scalability of algorithm for large number of inputs.

Asymptotic Analysis -

- Analyse the algorithm's performance

as input size approaches infinity.

Asymptotic Notations - Input size = n

1. Best Case : Ω - Minimum time required for program execution
lower bound : Running Time
2. Average Case : Θ - Average time required for program execution
tight bound (upper & lower)
3. Worst Case : O Big-oh - Maximum time required for program execution
Upper bound : Running time

Note : Worst-case analysis is most commonly used for guaranteed performance

Algorithm Complexity : 2 types.

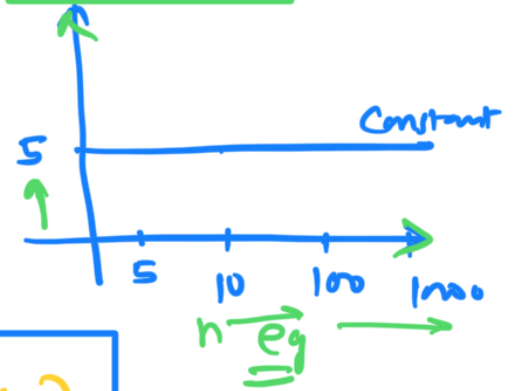
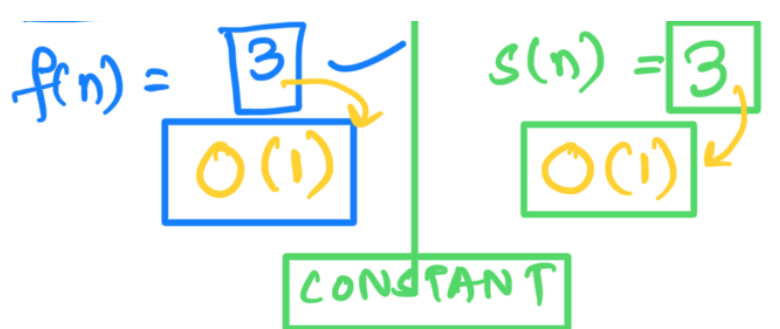
- Time Complexity
1. Time factor - Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.

- Space Complexity
2. Space factor - Space is measured by counting the maximum memory space required by the algorithm.

Ex: 1 Constant : $O(1)$

Algorithm Swap (a, b)

	Time	Space
{		
temp = a;	→ 1 ms	a → 1
a = b;	→ 1 ms	b → 1
b = temp;	→ 1 ms	temp → 1
}		



Logic: 1 line - 1 sec

$$x = 5x_a + 6x_b \rightarrow 1$$

$$x = 5x_a + 6x_b \rightarrow 1$$

$$x = 5x_a + 6x_b \rightarrow 1$$

$$x = 5x_a + 6x_b \rightarrow 1$$

$$f(n) =$$

4 → $O(1)$
 Constant
 Constant Time Complexity

Ex: 2

$$n = 5$$

$i = 0, 1, 2, 3, 4, \boxed{5}$ (false)
 size

A →

0	1	2	3	4
8	3	9	7	2

Algorithm Sum(A, n)

$\{$
 $s = 0;$ → 1
 $\text{for } (i = 0; i < n; i++)$ → $n+1$
 $\{$
 $s = s + A[i];$ → n
 $\}$
 $\text{return } s;$ → 1
 $\}$

Space

$A \rightarrow n$
 $n \rightarrow 1$
 $s \rightarrow 1$
 $i \rightarrow 1$

$2n + 3$
 ignore $2n$ → $O(n)$
 Highest
 ignore 3

$$f(n) = \underline{2n} + \underline{3}$$

$$\Downarrow$$

$$O(n)$$

$$s(n) = n + 3$$

$$\Downarrow$$

$$O(n)$$

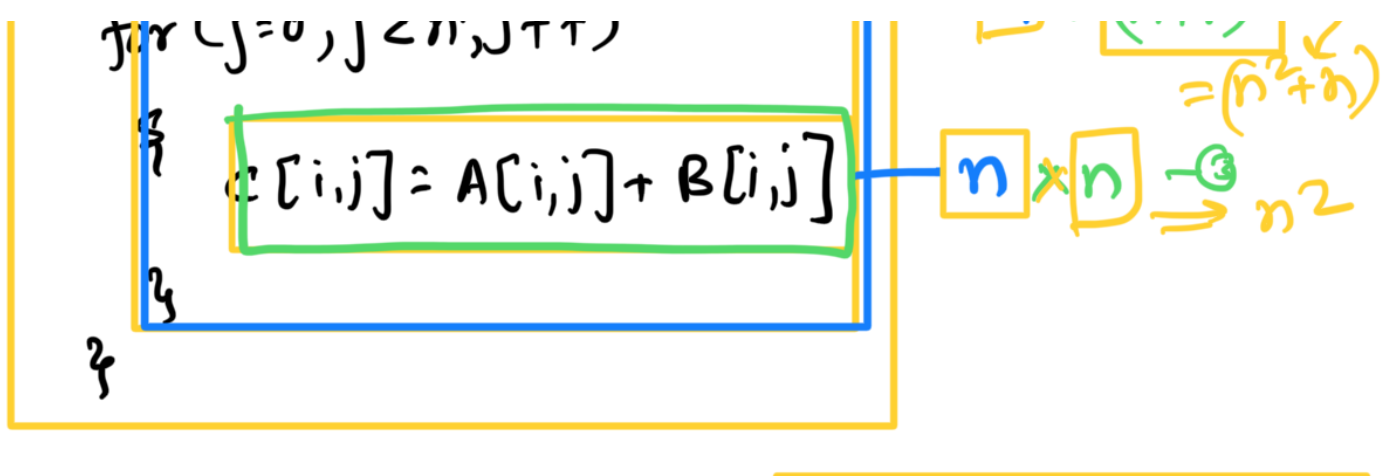
Linear Time Complexity

Ex: 3

Algorithm Add(A, B, n)

Time

$\{$
 $\text{for } (i = 0; i < n; i++)$ → $n+1$ — ①
 $\{$
 ... → $n \times (n+1)$ — ②



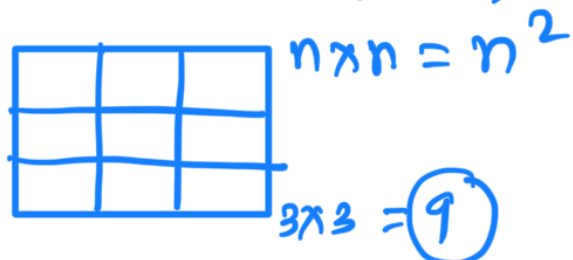
$$\begin{aligned} \textcircled{1} - n+1 &= n+1 \\ \textcircled{2} - n(n+1) &= n^2+n \\ \textcircled{3} - n \times n &= n^2 \end{aligned} \Rightarrow f(n) = (n+1) + n^2+n + n^2 = 2n^2 + 2n + 1$$

$$f(n) = \frac{2n^2}{\text{Quadratic}} + \frac{2n}{\text{Linear}} + \frac{1}{\text{Const}} \Rightarrow O(n^2)$$



Quadratic Time Complexity

Space $\rightarrow n \times n \Rightarrow n=3$ (3×3)



$$\begin{aligned} n \times n \\ n=3 \\ n \times n = n^2 \end{aligned}$$

$$A - n \times n = n^2$$

$$B - n \times n = n^2$$

$$C - n \times n = n^2$$

$$n - \text{—————} 1$$

$$i - \text{—————} 1$$

$$j - \text{—————} 1$$

$$S(n) = 3n^2 + 3 \approx O(n^2)$$

Ex: 4 Algorithm Multiply (A, B, n)

$$n+1 \text{ — } \text{for } (i=0, i<n; i++) \text{ — } O(n^3)$$

$$(n+1) \times n \text{ — } \{ \text{for } (j=0; j<n; j++) \text{ — } \}$$

$n \times n$ 1 $C[i,j] = 0;$

$(n+1) \times n \times n$ — `for (k=0; k < n; k++)` ~

$n \times n \times n$ — $\{$
 $\} = 2n^3 + 3n^2 + 2n + 1$ $\{$
 $C[i, j] = C[i, j] + A[i, k] * B[k, i]$

$f(n) = 2n^3 + 3n^2 + 2n + 1$

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$

$O(n^3)$

Spalte

$$A - nxn = n^2$$
$$\beta - n \times n = n^2$$

$C \sim n \times n = n^2$

n	_____	
i	_____	
j	_____	
k	_____	

$$\underline{S(n) = 3n^2 + 4 \approx O(n^2)}$$

Ex: $\{ \text{for } (i=0; i < n; i++) \quad \text{--- } n+1$
 $\{ \quad \text{st; } \quad \text{--- } n$
 $\}$ $\} O(n)$

Ex: 6 for (i=n; i > 0, i--) — n+1 } O(n)
 {
 st; ————— n
 }

Ex: 7 for ($i=1$; $i < n$; $i+=2$) — $\frac{n+1}{2}$ } $O(n)$
 {
 st; (i) → $\frac{n}{2}$

Ex 8 for ($i=1$; $i < n$; $i = i + 20$) — $n+1$? $O(n)$

{ st $\frac{n}{20}$ }

Ex: 9 for (i=0; i<n; i++) — $n+1$
 { for (j=0; j<n; j++) — $n \times n+1$
 { st; — $n \times n$
 }
 }
 $O(n^2)$

Ex: 10 for (i=0; i<n; i++) — $n+1$
 {
 {
 for (j=0; j<i; j++) — $n+1$
 {
 }
 }
 } $O(n)$

Ex: 11 for (i=0; i<n; i++)
 {
 for (j=0; j<i; j++)
 {
 st;
 }
 }
 i j No of Hm

0	0 x	0
1	0 ✓	1
	1 x	
2	0 ✓	2
	1 ✓	
	2 x	
3	0 ✓	3
	1 ✓	
	2 ✓	
	3 x	
⋮	0 ✓	
⋮	1 ✓	
⋮	2 ✓	
⋮	⋮	
⋮	n-1 ✓	
⋮	n x	

$f(n) = 1 + 2 + 3 + 4 + 5 + \dots + n$ Sum of natural numbers

$$f(n) = \frac{n^2 + 1}{2}$$

$O(n^2)$

Ex: 12 $0 = 0$ $\sqrt{n}$ i p

for (i=1; $p \leq n$; i++)

{ $p = p + i$ }

}

Assume,

$p > n$

opposite
Assume

— (1)

$$p = \frac{k(k+1)}{2}$$

— (2)

Equate 1, 2 $\Rightarrow \frac{k(k+1)}{2} > n$

$$k^2 > n$$

$$k > \sqrt{n} \approx O(\sqrt{n})$$

1 $0+1=1$

2 $1+2=3$

3 $1+2+3$

4 $1+2+3+4$

...

$k \quad 1+2+3+4+\dots+k$

$\underbrace{\hspace{1cm}}_P$
 $1+\dots+n$
 $\downarrow$
 $\frac{n(n+1)}{2}$

Square root Time Complexity

Ex:

for (i=1; $i < n$; $i = i * 2$)

{

st;

}

Assume

$i \geq n$

— (1)

$i = 2^k$

— (2)

$$2^k \geq n$$

$$2^k = n$$

$$k = \log_2 n \approx O(\log_2 n)$$

Logarithmic Time Complexity

Ex:

for (i=1; $i < n$; $i = i * 2$)

{

st;

$\rightarrow$

$O(\log_2 n)$

i

1 $1 \times 2 = 2$

2 $2 \times 2 = 2^2$

3 $2^2 \times 2 = 2^3$

4 $2^3 \times 2 = 2^4$

...

k $2^{k-1} \times 2 = 2^k$

$\underline{\underline{2^k}}$

Ex: $\{$ for ($i=n$; $i \geq 1$; $i = i/2$)
 $\{$ $\}$ $\}$ $\}$
 $O(\log_2 n)$

$$\frac{n}{2}, \frac{n}{2^2}, \frac{n}{2^3}, \dots, \frac{n}{2^k}$$

Ex: $\{$ for ($i=0$; $i \leq i < n$; $i++$)
 $\{$ $\}$ $\}$
 $O(\sqrt{n})$

$$i^2 < n$$

$$i = \sqrt{n}$$

Ex: $\{$ for ($i=0$; $i < n$; $i++$) $\}$ $n+1$
 $\{$ for ($j=1$; $j < n$; $j = j \times 2$) $\}$ $n \times \log n$
 $\{$ $\}$ $n \times \log n$
 $\}$
 $2n \log n + n$
 $\Downarrow$
 $O(n \log_2 n)$